/

Course Name:	**Competitive Programming Lab**	**Semester:**	**IV**
Date of Performance:	**18 / 02 / 2026**	**DIV/ Batch No:**	**A-3**
Student Name:	**Shrived Deshmukh**	**Roll No:**	**71**

Experiment No: 4

Title: Sliding Window Techniques

Aim of the Experiment: To understand and implement Sliding Window Techniques for a given problem

Objective of the Experiment:

After completing this experiment, the student will be able to:

- Understand the concept and working of the Sliding Window technique.
- Identify suitable problem scenarios for applying the Sliding Window approach.
- Implement Sliding Window algorithms efficiently using **C++**.
- Analyze the time and space complexity of the implemented C++ solution.
- Enhance algorithmic problem-solving skills through optimized C++ programs.

COs to be achieved:

CO 2: Analyse and optimize algorithms using amortized analysis and bit manipulation, equipping them to tackle complex computational problems.

Theory:

The Sliding Window technique is an efficient algorithmic approach used to solve problems involving contiguous subarrays or substrings. Instead of repeatedly recalculating values for every possible window, this technique maintains a window of elements and slides it across the data structure (array or string), updating the result incrementally. This significantly reduces time complexity compared to brute-force methods.

A sliding window is defined by two pointers, usually referred to as the **left** and **right** boundaries. As the right pointer moves forward, new elements enter the window, and computations are updated. Depending on the problem, the left pointer may also move forward to shrink the window and maintain certain constraints such as window size or validity conditions.

There are two main types of Sliding Window techniques:

1. **Fixed-size Sliding Window** – where the window size remains constant (e.g., finding the maximum sum of subarrays of size *k*).
2. **Variable-size Sliding Window** – where the window size changes dynamically based on problem constraints (e.g., longest substring without repeating characters).

In C++, Sliding Window algorithms are efficiently implemented using loops, pointers, and data structures such as arrays, strings, hash maps (`unordered_map`), or sets. This approach optimizes performance by reducing nested loops and lowering the overall time complexity from O(n2)O(n^2)O(n2) to O(n)O(n)O(n) in many cases.

Thus, the Sliding Window technique is widely used in solving real-world problems related to arrays, strings, and sequences, making it an essential concept for competitive programming and efficient algorithm design.

Working Mechanism

A typically Sliding Window technique follows these steps:

 Use two pointers (**start** and **end**) to define a window of contiguous elements.

 Initialize both pointers at the beginning of the array or string.

 Move the **end pointer** forward to expand the window by including new elements.

 Update the required computation (sum, count, frequency, etc.) incrementally.

 When the window size reaches a limit or a condition is violated, move the **start pointer** forward to shrink the window.

 Remove the effect of elements leaving the window while shrinking.

 Continue expanding and shrinking the window until all elements are processed.

 Achieve efficient performance by avoiding repeated recalculations, often resulting in **O(n)** time complexity.

Advantages

 Reduces time complexity compared to brute-force approaches.

 Efficient for problems involving **contiguous subarrays or substrings**.

 Avoids redundant calculations by reusing previous results.

- Simple to implement using two pointers in **C++**.
- Improves performance, often achieving **O(n)** time complexity.
- Widely applicable in array and string processing problems.

Limitations

- Applicable only to problems involving **continuous (contiguous) data**.
- Not suitable for problems requiring non-contiguous element selection.
- Can be tricky to design for **variable-size window** problems.
- Requires careful handling of window boundaries and conditions.
- May need extra data structures (maps/sets), increasing space usage.

Problem Statements:

Difficulty (as per lab)	Problem	LeetCode mapping
Easy	0/1 Knapsack (small constraints)	LC 643 – Maximum Average Subarray I
Medium	Matrix chain multiplication	LC 3 – Longest Substring Without Repeating Characters
Difficult	Edit distance	LC 76 – Minimum Window Substring

Code :

Q1-Easy-Knapsack(Small constraints)

Code

C++ Auto

```cpp
#include <vector>
#include <algorithm>
using namespace std;

class Solution {
public:
    double findMaxAverage(vector<int>& nums, int k) {
        int n = nums.size();
        double sum = 0;

        for (int i = 0; i < k; ++i)
            sum += nums[i];

        double maxSum = sum;

        for (int i = k; i < n; ++i) {
            sum = sum - nums[i - k] + nums[i];
            maxSum = max(maxSum, sum);
        }

        return maxSum / k;
    }
};
```

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =

```
[1,12,-5,-6,50,3]
```

k =

```
4
```

Output

```
12.75000
```

Expected

```
12.75000
```

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[5]

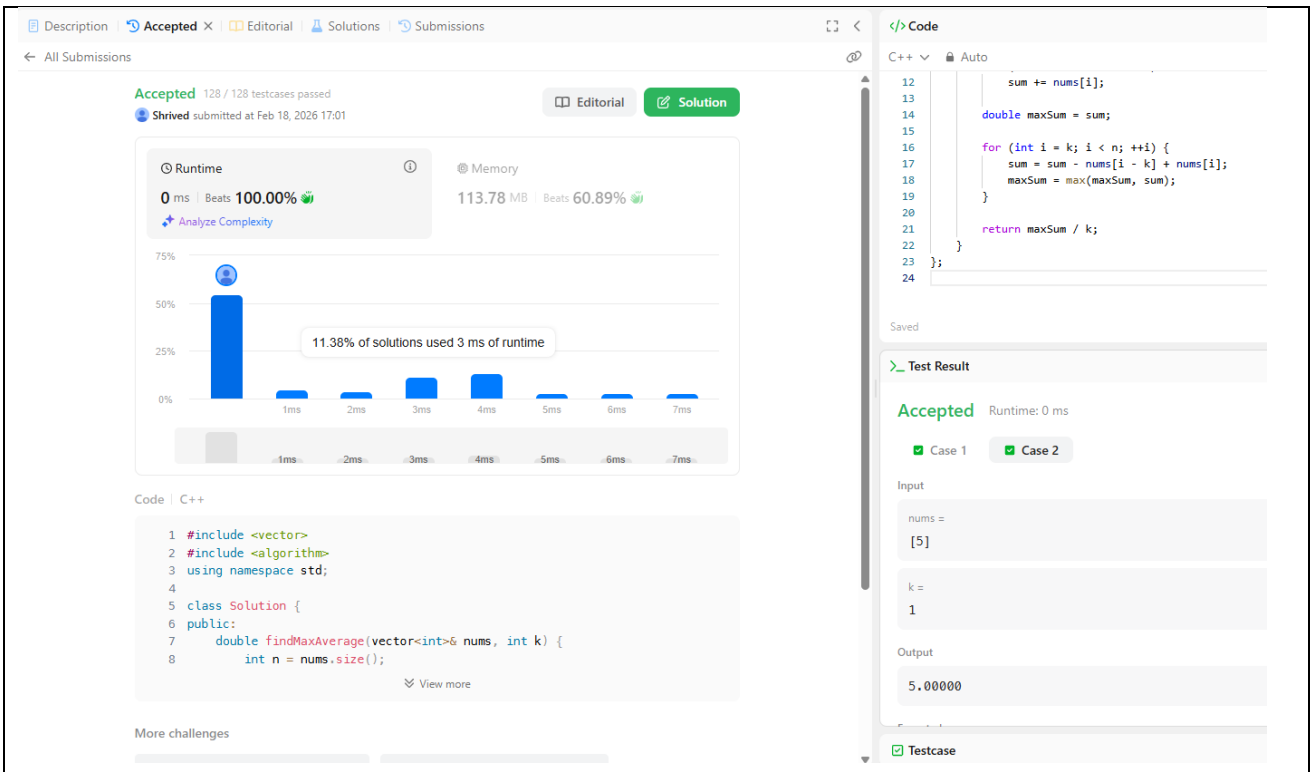
k =
1

Output

5.00000

Expected

5.00000



Q2-medium-Matric chain multiplication
Code:-

Code

C++ Auto

```cpp
#include <string>
#include <unordered_map>
#include <algorithm>
using namespace std;

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_map<char, int> lastIndex;
        int maxLen = 0, left = 0;

        for (int right = 0; right < (int)s.size(); ++right) {
            if (lastIndex.count(s[right]) && lastIndex[s[right]] >= left) {
                left = lastIndex[s[right]] + 1;
            }
            lastIndex[s[right]] = right;
            maxLen = max(maxLen, right - left + 1);
        }

        return maxLen;
    }
};
```

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s =
"abcabcbb"

Output

3

Expected

3

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s =
"bbbbb"

Output

1

Expected

1

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s =
"pwwkew"

Output

3

Expected

3

Description | Accepted | Editorial | Solutions | Submissions

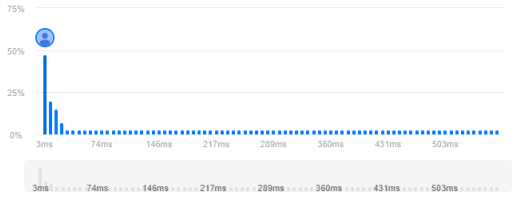
All Submissions

Accepted 988 / 988 testcases passed
Shrived submitted at Feb 18, 2026 17:05

Editorial Solution

Runtime
7 ms | Beats 62.78%
Analyze Complexity

Memory
11.72 MB Beats 63.93%



Code | C++

```
#include <string>
#include <unordered_map>
#include <algorithm>
using namespace std;

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
```

View more

More challenges

Code

C++ Auto

```
#include <string>
#include <unordered_map>
#include <algorithm>
using namespace std;

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_map<char, int> lastIndex;
        int maxLen = 0, left = 0;

        for (int right = 0; right < (int)s.size(); ++right) {
            if (lastIndex.count(s[right]) && lastIndex[s[right]] >= left) {
                left = lastIndex[s[right]] + 1;
```

Saved

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s =
"pwwkew"

Output

3

Expected

3

Testcase

Q3)Hard:-Edit Distance

Code

C++ Auto

```cpp
#include <string>
#include <climits>
using namespace std;

class Solution {
public:
    string minWindow(string s, string t) {
        int need[128] = {0}, window[128] = {0};
        for (char c : t) need[c]++;

        int left = 0, right = 0, valid = 0, required = 0;
        for (int i = 0; i < 128; ++i)
            if (need[i] > 0) required++;

        int start = 0, minLen = INT_MAX;

        while (right < (int)s.size()) {
            char c = s[right++];
            if (need[c] > 0) {
                window[c]++;
                if (window[c] == need[c]) valid++;
            }

            while (valid == required) {
                if (right - left < minLen) {
                    start = left;
                    minLen = right - left;
                }

                char d = s[left++];
                if (need[d] > 0) {
                    if (window[d] == need[d]) valid--;
                    window[d]--;
                }
            }
        }

        return minLen == INT_MAX ? "" : s.substr(start, minLen);
    }
};
```

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s =
"ADOBECODEBANC"

t =
"ABC"

Output

"BANC"

Expected

"BANC"

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s =
"a"

t =
"a"

Output

"a"

Expected

"a"

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s =
"a"

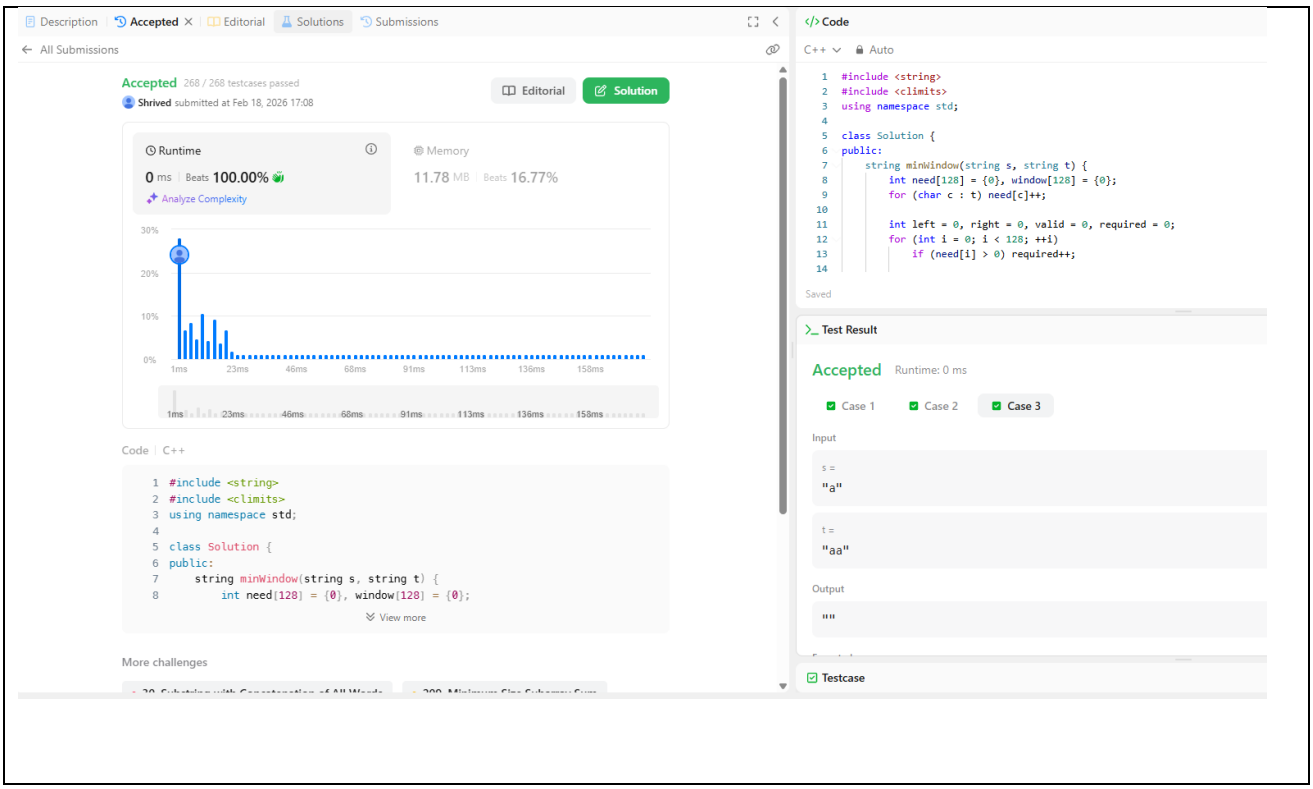
t =
"aa"

Output

""

Expected

""



Output:

Post Lab Subjective/Objective type Questions:

1. Explain how Sliding Window ensures **O(n)** time complexity even though two pointers are used.

   Sliding Window ensures O(n) time complexity because both pointers—left and right—only move forward through the input without ever moving backward. Although the algorithm uses nested loops conceptually, each element is visited at most twice: once when the right pointer expands the window and once when the left pointer shrinks it. This means the total number of pointer movements is at most 2n, making the overall process linear in time relative to the input size.

2. Given an array containing negative numbers, does the Sliding Window technique always work? Explain your answer.
   Sliding Window technique does not always work on arrays containing negative numbers. This is because sliding window relies on the property that when you move the window boundaries, the sum or value inside the window changes predictably—usually increasing or decreasing steadily.

With negative numbers, adding or removing elements can cause the sum to fluctuate unpredictably, making it difficult to determine when to expand or shrink the window correctly. Therefore, sliding window works best for problems where the values are non-negative or where the problem guarantees a monotonic behavior in the window, but it may fail or require modifications when negative numbers are involved.

Conclusion:

In conclusion, the Sliding Window technique is a powerful tool for efficiently solving problems involving contiguous subarrays or substrings, providing linear time complexity when elements are processed in a predictable manner. However, its effectiveness depends on the properties of the data, and it may not work directly for arrays with negative numbers or cases where the window's value does not change monotonically. Understanding when and how to apply sliding window is key to using it correctly and optimizing performance.

Signature of faculty in-charge with Date: